



Transcript

Week 26: Transformers

Video 1 - Introduction to Transformers

Ever wondered how A-I understands language with such accuracy?

Transformers are the reason behind breakthroughs in tools like Chat G-P-T and Google Translate.

This week, you'll discover how they learn patterns, words, and meaning—without being told exactly what to look for.

Let's explore how this model has reshaped the future of machine learning.

Transformers are a major breakthrough in A-I, introduced by Vaswani and colleagues in 2017, in a paper titled *Attention Is All You Need*.

They helped solve a key problem in natural language processing—how to understand long sentences with many words that relate to each other in complex ways.

Instead of reading words one by one, like Recurrent Neural Networks—or R-N-Ns—transformers look at all the words at once.

This is called parallel processing, and it makes the model faster and more efficient.

The core idea behind transformers is the attention mechanism.

It helps the model figure out which words in a sentence are most important.

For example, in the sentence “The cat sat on the mat,” attention highlights how “sat” connects to both “cat” and “mat.”

They also use self-attention, which means comparing every word with every other word in the sentence to understand relationships and context.

This makes the model better at tasks like translation, summarisation, and conversation.

These features power advanced tools like Chat G-P-T, BERT, and Google Translate.

Transformers brought major improvements to how neural networks work.

First, they are excellent at understanding context—even when the related words are far apart in a sentence.

For example, in the sentence, “The book she bought last week is on the table,” transformers can connect “book” and “table” even though they are far apart.

This is something older models like R-N-Ns, or Recurrent Neural Networks, often struggle with.

Second, transformers use parallel processing.

That means they can look at the whole input at once, instead of one step at a time.

This makes training much faster, and allows the model to scale to very large datasets.

Finally, transformers are not just for language.

They work well in a variety of applications—including speech recognition, image generation, translation, and even generative tools like Chat G-P-T.

These three features—context understanding, efficient training, and versatile use—make transformers the foundation of today's most powerful A-I systems.

Transformers have rapidly transformed the A-I landscape in just a few years.

It began in 2017, when researchers Vaswani and colleagues introduced the original transformer model in a paper titled *Attention Is All You Need*.



This laid the foundation for a new class of neural networks—capable of understanding language with far greater accuracy.

In 2018, models like BERT and G-P-T took the field by storm. They brought huge improvements in natural language processing—powering tools for search, chat, and summarisation.

By 2020, the same transformer approach extended into computer vision with Vision Transformers. This showed that the model wasn't just for text—it could also recognise images and patterns visually.

From 2023 onward, we've entered the era of multi-modal A-I—where a single model can process language, images, and even audio together. This includes advanced systems like G-P-T4 and Gemini, which combine different data types to generate rich, context-aware responses.

The transformer's evolution shows how one idea—attention—sparked a wave of innovation across all areas of A-I.

Transformers are at the core of many powerful A-I systems used today.

In natural language processing, they help models understand and generate human language. This includes tasks like translation, summarisation, and answering questions—key features in apps like Google Translate and A-I-powered search engines.

In computer vision, Vision Transformers—or V-i-Ts—apply the same architecture to images. These models can identify objects, analyse scenes, and support technologies like self-driving cars and medical image diagnostics.

In speech processing, transformers power tools like Whisper, which transcribe spoken language into text with high accuracy—even across multiple languages or accents.

And finally, transformers are at the heart of generative A-I. They enable models like Chat G-P-T for conversation, DALL-E for image generation from text, and other creative tools that blend words, visuals, and sound.

Together, these use cases show how transformers extend far beyond language. They now shape how machines read, listen, see, and create.

Transformers are no longer just one innovation—they've become the foundation for much of modern A-I.

At the core, they power today's most influential tools—like Chat G-P-T, DALL-E, and other advanced language and image models.

They've unlocked multi-modal A-I, where a single system can work with text, images, audio, and more. This is what allows models to describe a photo, answer a question about it, and even generate related visuals.

Because transformers process all inputs at once, they are also highly scalable. This makes it possible to train them on enormous datasets with millions of examples—critical for large language models.

And perhaps most importantly, transformers represent the future of A-I.



From personalised assistants to scientific discovery, their impact is expanding across every industry.

So, when we talk about cutting-edge A-I, we're really talking about the ripple effect of transformers.

How might you apply transformers in your own domain—whether it's language, vision, or beyond?

The next breakthrough could start with your idea.

Video 2 - Transformers: Core Concepts

Have you ever wondered how A-I systems like Chat G-P-T understand text, translate languages, or generate images so well?

The secret lies in a model called the transformer.

In this topic, we'll explore what makes transformers so powerful—and why they're now the foundation of today's most advanced A-I tools.

Let's explore the main components that make up a transformer model.

These are the parts that help the model process information quickly and understand complex input sequences.

First, we have the encoder layers, which handle the input.

These layers are repeated N times to help the model learn deep patterns.

Next are the decoder layers, also repeated N times.

These generate output—like translated text or predicted tokens.

Then comes the self-attention mechanism, which helps the model figure out which parts of the input matter most.

When combined with multi-head attention, the model can focus on several parts of a sentence at the same time.

Feed-forward networks add extra learning steps inside each encoder and decoder block.

Positional encoding gives the model a sense of word order—because, unlike R-N-Ns, transformers don't read words one by one.

Finally, we have residual connections and layer normalisation.

These help stabilise training and make learning faster.

Together, these components allow transformers to generate sequences efficiently and effectively.

Let's explore the encoder layer—one of the core components in a transformer model.

The encoder processes the input sequence, like a sentence, and this structure is repeated N times.

Repeating it helps the model deeply understand patterns and relationships between words.

Each encoder block contains four essential parts.

The first is the self-attention mechanism.

This allows the model to examine every word in the sentence in relation to every other word.

For example, in the sentence *"The book she bought last week is on the table,"*



the word “*book*” is related to “*table*” even though they are far apart. Self-attention helps the model make that connection. This is what gives transformers their ability to capture long-range dependencies in text.

Next is the feed-forward neural network, also called F-F-N-N. After self-attention identifies relationships, the F-F-N-N transforms each word’s representation. It applies two layers of small neural networks with a non-linear activation function in between. This helps the model refine the meaning of each word based on its context.

Then we have the residual connections. You can think of these as skip connections. Instead of just sending the output forward layer by layer, the residual path allows the original input to be added back into the output. This prevents the model from forgetting important information and helps it train more smoothly—especially in deep networks.

Finally, there’s layer normalisation. During training, the numbers flowing through a network can become unstable—either too large or too small. Layer normalisation fixes this by scaling the activations to have zero mean and unit variance. This stabilises the training process and helps the model converge faster.

All these components—self-attention, F-F-N-N, residuals, and normalisation—work together in each encoder layer. They allow the model to process text efficiently, with a clear understanding of word meaning and relationships.

In the upcoming slides, we’ll break down each of these parts with diagrams and examples so you can see how they work in practice.

Let’s walk through how the encoder layer processes input data—step by step.

It all begins with raw text input—like a sentence or document. First, the model converts these words into numbers using input embeddings. These embeddings turn each word into a dense vector that the model can understand.

Next comes positional encoding. Because transformers don’t read text in order like R-N-Ns, we add positional information to help the model know where each word appears in the sentence. This allows it to understand both the content and the structure.

Then, we apply self-attention. This powerful step helps the model figure out which words are most relevant to each other. For example, in the sentence “*The keys are on the table,*” the model learns to connect “keys” with “table” because of their relationship—no matter how far apart they are in the sentence.

After that, we use residual connections and layer normalisation—a step often labelled *Add & Norm*. Residual connections help the model keep important information flowing forward. Layer normalisation makes sure that values don’t explode or vanish during training, which helps the model learn faster and more reliably.



Finally, the data moves through a feed-forward neural network, or F-F-N-N. This step applies additional transformations to each word's representation, refining the information even further. It uses two small layers with a non-linear function called ReL-U in between—spelled RELU.

Together, these steps transform plain text into rich, contextualised data. This refined output is now ready for deeper layers—or for decoding into something new, like a translation or summary.

Let's look at how the decoder works—this is the part of the transformer that generates output, like a translation or summary.

The decoder also repeats the same structure N times, and each block has three main components.

First, we have masked self-attention.

This step ensures the model doesn't "look ahead" at future tokens during training. For example, if the model is generating a sentence word by word, it should only focus on the words it has already generated. To make this happen, we apply a mask that blocks the model from seeing future positions. This is often called causal attention because it respects the order of the sequence.

Next is cross-attention.

This is where the decoder connects with the encoder's output. In this step, the decoder checks which parts of the original input—like a source sentence—are most relevant for generating each word in the output. It matches the encoder's hidden states with the decoder's current state, helping it generate more accurate and aligned responses.

Then comes the feed-forward neural network, or F-F-N-N.

Just like in the encoder, this part applies two small layers with a ReL-U activation function in between.

This helps to refine the meaning of each token even further.

We also apply residual connections and layer normalisation, labelled together as *Add & Norm*.

These steps ensure that training is smooth and stable.

They prevent gradients from vanishing or exploding, especially when stacked across many layers.

All these parts work together so the decoder can generate sequences one token at a time—using both what it has already produced and what it learned from the input.

In the next section, we'll see how these decoder outputs are turned into words, sentences, and useful results.

The decoder layer transforms hidden representations into meaningful output—one token at a time.

Let's walk through each step in the process.

We begin with input embedding, where each token is converted into a numerical vector the model can work with.

Next comes positional encoding, which adds information about the order of the tokens. Since transformers don't read words in order like R-N-Ns or L-S-T-Ms, this step helps the decoder track where each token appears.

Then we apply masked self-attention.

This layer ensures the decoder only sees the tokens it has already generated—not any



tokens ahead.

For example, when generating the third word, the model can only use the first and second—not the fourth or fifth.

We call this *causal attention* because it preserves the natural flow of information, just like we read or write a sentence.

Next, we use Add & Norm—a combination of residual connection and layer normalisation. The residual path adds the original input back into the layer's output to prevent information loss.

Layer normalisation scales the values to keep training stable and fast.

After that, the decoder reaches out to the encoder using cross-attention.

Here, it compares the current decoder state with the encoder's output.

For example, if the encoder processed a sentence in French, cross-attention helps the decoder focus on the right French words while generating an English translation.

This alignment step links the decoder's hidden states with the encoder's—ensuring accuracy.

We apply Add & Norm again to stabilise this output.

Then comes the feed-forward neural network, or F-F-N-N.

This layer applies two linear transformations with a ReLU activation in between.

It further processes each token's representation, adding clarity and structure.

Finally, we apply a third Add & Norm.

This ensures the output is smooth and ready for the next decoder layer—or the final output step.

All together, these steps allow the transformer decoder to generate high-quality sequences, one token at a time, while staying grounded in the input context.

Self-attention is a core feature of transformers.

It allows the model to look at all the words in a sentence—and decide which ones matter most to each word being processed.

To understand how this works, let's walk through the process step by step.

Step 1: Q, K, and V vectors

We begin by creating three vectors for every word in the sentence.

These are called the Query, Key, and Value vectors—written as Q, K, and V.

These vectors are learned from the input data using weight matrices.

The query asks, "What am I looking for?"

The key answers, "What do I have to offer?"

And the value carries the actual content that may be passed forward.

Next, we move to Step 2—the dot product.

This step compares the query of one word with the keys of all the other words.

We use a mathematical operation called the dot product to measure similarity.

A higher dot product means the two words are more closely related.

This tells the model how much attention to pay to each word.

Now that we've measured similarity, we go to Step 3—applying Softmax.

The Softmax function turns those similarity scores into attention weights.

These weights behave like probabilities and determine how much focus each word should receive.

The higher the attention weight, the more influence that word has on the final result.



Finally, we come to Step 4—the weighted sum.

Here, each value vector is multiplied by its attention weight, and all of them are summed together.

This creates a new vector that blends information from the most relevant words. Each word ends up with a custom, context-aware representation.

To summarise, self-attention helps the model focus on the right parts of the sentence—no matter how close or far the words are.

This makes it incredibly powerful for tasks like translation, summarisation, and question answering.

Multi-head attention is one of the most powerful innovations inside the transformer architecture.

It builds on the idea of self-attention, but instead of using just one attention mechanism, it runs multiple attention heads in parallel.

Let's walk through what that means—and why it matters.

To start, imagine that each attention head is like a spotlight, focusing on a different part of a sentence.

Instead of using a single spotlight for the entire sequence, multi-head attention uses several at once.

Each head has its own set of learned parameters, meaning it can learn to focus on something unique.

For example, one head might learn to spot short-range dependencies, like a word and its immediate neighbour.

Another might focus on long-range relationships, like connecting the beginning of a sentence to the end.

This variety is called diverse representation, and it allows the model to understand language in a deeper, more flexible way.

Once each head finishes its self-attention process, the outputs are concatenated—which means they are stacked together into one long vector.

This combined output brings in multiple viewpoints and gives the model a richer understanding of the input.

But we don't stop there.

This concatenated output is passed through a linear projection layer.

This step compresses all that information down to a usable size, so it can be passed to the next layer in the model.

So how does this help the model learn better?

By letting each attention head specialise in a different aspect of the input, the transformer captures relationships at multiple levels.

It can learn both the grammar and the meaning of a sentence at the same time.

This is especially useful in complex tasks like translation, where one part of the model might need to match words, while another might focus on overall sentence structure or cultural context.

To sum up, multi-head attention is like giving the model a set of lenses to look at the same input from different angles.

It helps the transformer model process information more effectively, build richer representations,

and ultimately perform better on a wide range of tasks—from summarisation to code generation.



Transformers are powerful because they process all tokens in parallel. But unlike RNNs or LSTMs, they don't read tokens in sequence. That means they have no built-in sense of word order.

For language tasks like translation, question answering, or summarisation, word order is essential.

So how do we give transformers that sense of sequence?

The answer is positional encoding.

Positional encoding is a method that helps the model track where each word appears in a sentence.

We do this by adding a special position vector to every token's embedding.

This gives each word not only its meaning—but also a signal about its position in the sentence.

Now let's look at how that's calculated.

For any position—spelled pos —and dimension—spelled i —the model uses sine and cosine formulas.

If the dimension index is even:

We apply the sine function to the position divided by ten thousand raised to the power of i divided by d .

Here, d is the total number of dimensions in the model.

If the dimension index is odd:

We apply the cosine function to the same expression:

Position divided by ten thousand raised to the power of i divided by d .

This gives each position a unique signature that repeats smoothly across dimensions.

The repeating nature of sine and cosine lets the model calculate relative distances between tokens.

For example, it can tell how far apart the words "The" and "Table" are—even if several words come between them.

Without positional encoding, the model would treat a sentence as a bag of words, ignoring order completely.

By adding this signal, we make it possible for transformers to reason about sequence—while still enjoying the speed of parallel processing.

As transformer models grow deeper, they face two big training challenges—one is vanishing gradients, and the other is unstable activations.

To tackle these, transformers use two techniques: residual connections and layer normalisation.

Let's start with residual connections, also called skip connections.

These create shortcut paths that allow information—and gradients—to flow directly across layers.

Instead of just passing output from one layer to the next, the model adds the input of the layer back into the output.

This way, important information is preserved—even across many layers.

Why does this help?

Because it prevents gradients from shrinking to zero during training.

This improves training stability and makes it easier for the model to learn, especially in deep networks.



Now let's talk about layer normalisation.

After the residual step, the model applies a normalisation function to the combined output. It adjusts the values so they have a mean of zero and a variance of one. This helps to stabilise the network and ensures consistent performance.

Why is this important?

Without normalisation, activations might explode or vanish, causing training to break down.

With it, the model trains faster, makes fewer mistakes, and adapts better to complex patterns.

Together, these two tools—residual connections and layer normalisation—help transformers stay stable, efficient, and accurate, even as they process large amounts of data across dozens of layers.

After the attention layers have done their job—helping the model understand relationships between words—we need to take the next step.

That step is refinement.

We want to strengthen and clarify each token's meaning before it moves forward in the model.

That's exactly what the feed-forward neural network, or FFNN, is designed to do.

This part of the transformer operates independently on every token.

Unlike attention layers that compare words with one another, the F-F-N-N looks at each word on its own.

It takes the representation of a single token and processes it further, giving the model more flexibility and depth.

So how does it work?

The F-F-N-N includes two layers.

In the first layer, the model applies a linear transformation.

This means it multiplies the input vector by a matrix of learned weights and adds a bias term.

That operation reshapes the input, allowing the model to explore new combinations of features.

But if we stopped there, the model would only learn straight-line relationships—simple and limited.

To go beyond that, we introduce a non-linear activation function.

The most common one used here is called ReL-U, which stands for Rectified Linear Unit. Spelled RELU, this function replaces all negative values with zero and keeps the positive ones as they are.

It might sound simple, but ReL-U helps the model learn non-linear patterns—like curves and switches—rather than just flat trends.

This is what lets the transformer understand things like sarcasm in text, unusual phrasing, or flexible syntax.

Once the ReLU has done its part, we move to the second layer, which applies another linear transformation.

This step adjusts the size of the output so it fits neatly with the model's expected input dimensions for the next layer.

It's like reshaping the information one more time before handing it off to the next processing stage.

Even though it's just two steps, the F-F-N-N plays a critical role in the transformer.



It lets the model refine what it has learned about each token, giving it sharper meaning and better readiness for tasks like prediction, translation, or summarisation.

So while attention layers help the model see how words relate to one another, the F-F-N-N makes sure each word is internally well-structured—ready to move forward in the stack.

This combination of global attention and local refinement is part of what makes transformers so powerful and adaptable.

Once the decoder has processed the input, it's ready to generate the final output. This is the last step in the transformer pipeline—where predictions are actually turned into language.

For tasks like translation, summarisation, or language modelling, the decoder generates an output sequence—one token at a time. At each step, it predicts the next token based on everything it has seen so far.

But how does the model decide which token to choose?

That's where the softmax layer comes in.

This layer takes the decoder's raw output and converts it into a probability distribution. In other words, it tells us how likely each token in the vocabulary is to be the next word. The token with the highest probability is selected and added to the output sequence.

Let's look at an example.

Imagine the input is a prompt like: "The weather today is..."

The decoder might assign the highest score to the word "sunny," which means "sunny" will be selected as the next token in the sequence.

This process then repeats—using the newly generated word as part of the context for predicting the next one.

It continues until the model reaches an end token or hits a length limit.

In language modelling, this helps the model generate fluent, coherent sentences from scratch.

And in sequence-to-sequence tasks like translation or summarisation, the decoder maps the processed input into a meaningful output—word by word.

In short, the final output generation step is where everything comes together—the model's understanding of the input, its attention over context, and its learned language skills—to produce clear, useful results.

How might you apply what you've learned about transformer outputs to real-world challenges—like generating content, translating languages, or building smart assistants? Now that you understand how predictions are made, think about how you might improve or adapt this process in your own projects.

Video 3 - Transformer Models

Have you ever thought about how AI models decide what word comes next—or how they understand meaning at all?

Transformer models made this possible by changing how machines process information. Let's break down how they work, and why they're used in everything from chatbots to translation engines.



BERT—short for Bidirectional Encoder Representations from Transformers—is one of the most important models in natural language understanding.

What makes BERT unique is how it reads text.

Unlike older models that only read from left to right, BERT processes words in both directions at once.

This bidirectional approach helps the model understand the full context of each word in a sentence.

To get there, BERT goes through two key stages: pre-training and fine-tuning.

During pre-training, BERT learns language patterns using two tasks.

First, it uses masked language modelling, where random words in a sentence are hidden, and the model is asked to guess them.

For example, in the sentence “The cat sat on the [MASK],” BERT learns to predict the missing word using clues from both the left and the right.

Second, it uses next sentence prediction, where the model is given two sentences and must decide whether the second one logically follows the first.

These two tasks help BERT develop a deep sense of how language works.

Once pre-training is complete, BERT can be fine-tuned for specific jobs—like named entity recognition, sentiment analysis, or question answering.

In each case, BERT adapts to the task while keeping the strong foundation it learned during pre-training.

In summary, BERT is a powerful transformer model that understands language context from both sides.

And because it can be fine-tuned for many tasks, it’s widely used in real-world AI systems, from chatbots to research tools.

G-P-T, or Generative Pre-trained Transformer, is a model built for natural language generation.

Unlike BERT, which understands text in both directions, G-P-T reads and generates text from left to right.

It’s designed to predict the next word in a sentence, based only on the words that came before.

G-P-T uses an autoregressive approach.

That means it generates one word at a time, and each new word depends on the ones already generated.

This step-by-step process helps the model create logical and fluent output—whether it’s completing a sentence, answering a prompt, or writing an entire article.

G-P-T models are also built to scale.

As we move from G-P-T2 to G-P-T3 and G-P-T4, the models become more powerful by training on larger datasets and using more parameters.

This scaling allows G-P-T to handle more complex tasks, generate better responses, and adapt across different domains.

The model is trained in two stages.

First, it undergoes pre-training, where it learns general language patterns from a huge volume of text.

Then it’s fine-tuned for specific use cases, like code generation, chatbot conversations, or academic writing.

In short, G-P-T generates language by building on what it’s already written—word by word—making it a key tool for modern AI systems that create, complete, or summarise text.



Vision Transformers, or V-i-Ts, take the powerful transformer model—originally built for text—and apply it to images.

Instead of looking at an image all at once, V-i-Ts divide it into smaller, fixed-size patches. Each patch is treated like a word in a sentence. This is known as patch-based input.

Each patch is then flattened and linearly embedded into a vector, so the model can process it as structured data.

This step is called patch embedding, and it allows V-i-Ts to represent visual information in a format transformers can work with.

What makes V-i-Ts especially powerful is their use of self-attention.

Just like in text models, self-attention helps the model understand relationships—except now, it's between different parts of the image.

This means V-i-Ts can capture long-range spatial patterns across the image more effectively than traditional models.

V-i-Ts are also built to scale.

As we feed them more data and increase the number of parameters, their performance improves significantly.

In many tasks, they can even outperform convolutional neural networks, or CNNs.

So, by adapting core transformer ideas—like tokenisation, embedding, and attention—V-i-Ts bring a whole new level of flexibility and accuracy to image classification.

T-5, or the Text-to-Text Transfer Transformer, is a model that simplifies how we approach a wide range of natural language processing tasks. Whether it's translation, summarisation, classification, or question answering—T-5 reframes every task as a text-to-text problem. That means both the input and output are treated as plain text.

For example, if you want to translate a sentence, you simply feed it to the model with a prefix like “translate English to French,” and it generates the translated sentence as text. This format removes the need for specialised task-specific models.

T-5 is first pre-trained on massive datasets to help it learn general language patterns.

Then it is fine-tuned on specific tasks to improve accuracy and performance.

What makes T-5 especially powerful is its flexibility. With one unified framework, it can switch between multiple N-L-P tasks without redesigning the architecture each time. This makes T-5 ideal for multi-task learning environments, including academic, enterprise, and research settings.

X-L-Net is a powerful language model that combines the strengths of both BERT and GPT. It goes beyond traditional approaches by using a unique training method called permutation-based language modelling.

Instead of masking a word and predicting it, X-L-Net reorders the input in different ways—like shuffling the sequence—and learns to predict each word based on all possible positions. This lets it capture stronger and more flexible dependencies in the text.

Unlike BERT, which can only see the words before or after a masked token, X-L-Net sees the whole context in different orders, giving it a more complete view of meaning. At the same time, it keeps the benefits of GPT's autoregressive generation—meaning it can still generate high-quality text.

This combination makes X-L-Net especially effective in natural language tasks that require deep understanding, like reading comprehension or long-form question answering.

RoBERTa and Distil-BERT are two optimised variants of the original BERT model—each tailored to improve performance in different ways.

Let's start with RoBERTa. RoBERTa stands for *Robustly Optimised BERT Pretraining Approach*. It removes a step from BERT's training called Next Sentence Prediction, or N-S-P, which was found to add little value in many natural language understanding tasks. By removing this step, RoBERTa simplifies the training process and improves overall model accuracy. It also uses larger training datasets, bigger batch sizes, and more training time.



These enhancements make RoBERTa more robust and accurate, especially on benchmark NLP tasks like sentence classification and question answering.

Now, let's look at Distil-BERT. Distil-BERT is designed to be smaller, faster, and more efficient—without losing much performance. It keeps about 97 percent of BERT's accuracy, while using fewer parameters and requiring less computation. That means it's especially useful for real-time applications, or on devices with limited processing power. You'll often find Distil-BERT used in mobile apps, chatbots, or systems where speed is critical and energy use must be low.

In summary, RoBERTa pushes for maximum accuracy through more intensive training, while Distil-BERT prioritises speed and efficiency. Both show how modifying BERT can help meet specific needs—from high-performance tasks to resource-constrained environments.

Transformer models have transformed the field of artificial intelligence by powering a wide range of tasks across natural language processing and computer vision. Let's look at how different models are applied in real-world scenarios.

BERT is widely used for understanding text. It excels in tasks such as question answering, identifying named entities like people or organisations, classifying text into categories, and analysing sentiment.

G-P-T focuses on generating text. It performs tasks like text completion, writing stories, powering conversational agents, and even generating computer code. Its left-to-right prediction makes it well suited for sequential content generation.

Vision Transformers, or V-i-Ts, bring the transformer architecture into the visual domain. Instead of text, they work with images—breaking them into patches and processing them like tokens. This makes V-i-Ts ideal for image classification, object detection, and image segmentation.

T-5 treats every task as a text-to-text problem. Whether it's summarising a document, translating from one language to another, or answering a question, T-5 reframes it all as generating a new piece of text based on input text.

RoBERTa, a refined version of BERT, improves training by removing the Next Sentence Prediction step. It performs especially well on tasks like question answering, sentiment analysis, and text classification.

Distil-BERT is a smaller, faster version of BERT that maintains much of its accuracy. It's often used when speed and efficiency are critical, such as real-time text classification, sentiment analysis, and named entity recognition.

Together, these models show how transformer architectures can be adapted and optimised for a wide variety of tasks—spanning both language and vision.

Video 4: Life Without Transformers- Demo

Hey learners, let's get a good idea about what is life without transformers, meaning we have gone through how it has revolutionised the way NLP tasks have been performed, but just to get a good intuitive understanding that how transformers has really been a game-changer. Just to summarise everything that you have gone through. Let's directly compare the old models like RNNs and LSTMs with the new world of transformers.

If you talk about processing style, RNNs read text one word at a time like a slow reader



going through a novel word by word. Transformers on the other hand can look at the entire sentence at once. That makes them much faster and smarter in understanding context.

Memory of the context. RNNs often forget what happened earlier in a long sentence or paragraph. For example, if the sentence is I grew up in France, I speak fluent.

RNNs might lose track of France. Transformers use self-attention which allows them to remember and connect distant words easily. Training speed.

RNNs are slow because everything must happen step by step. Transformers can train in parallel which makes them dramatically faster and scalable. This is why models like GPT-4 are even possible today.

Applications. RNNs were mostly used for text tasks like early chat bots or translators. Transformers go far beyond.

They power translation, summarization, image recognition, speech-to-text and even generate AI like chat, GPT and DALY. Scalability. RNNs don't scale well.

When the input gets longer, they lose accuracy. Transformers, however, get better with more data and bigger architectures which is why scaling laws works so well for them. Real-world applications.

Without transformers, we would still have clunky translators and scripted chat bots. With transformers, we now have chat GPT, we have Whisper for speech, Vision Transformers for images and creative tools like DALY. Before transformers came along, the workhorses of natural language processing were rectal neural networks or RNNs and they are improved cousins, I would say, long short-term memory networks or LSTMs.

These models process text in a very sequential way, one word at a time. Imagine reading a book letter by letter, word by word and only moving forward after you have fully processed the previous one. That's how RNNs work that we discussed.

The problem with this approach was that it made learning slow and inefficient. Training couldn't be parallelised, meaning we couldn't take full advantage of modern computing power like your GPUs, TPUs and all. The problem with this approach was that it made learning slow and inefficient, as I mentioned.

Even worse, RNNs and LSTMs struggled with long-term dependencies. They often forgot information that came earlier in a long sentence or paragraph. Like the example I gave earlier, I grew up in France, I speak fluent.

By the time it reached the blank, it might have already forgotten the word France. On top of that, these models suffered from memory constraints. They could only handle short sequences effectively, which limited their real-world applications.

So while RNNs and LSTMs were groundbreaking for their time, they came with serious bottlenecks that made it hard to build large, reliable AI systems. Without a new approach, progress in natural language processing would have remained slow and limited. One of the biggest challenges with RNNs was the vanishing context problem.

Let's look at the simple example that I keep giving. I grew up in France, and then you might have some more sentences, and then I speak fluent, and we need the model to predict what word will come. The correct answer is obviously French, but older models often failed here.

Why? Because by the time they reached the blank, they had forgotten the word France.



Their memory of earlier parts of the sentence had faded away. This happened because RNNs processed text step by step, passing information forward like a game of telephone.

With every step, the original signal got weaker and weaker. This is called the vanishing gradient problem. The result was that important context from earlier in a sentence or even earlier in a paragraph was frequently lost.

That made these models unreliable for real-world tasks like translation, question answering, or long-form text generation. In short, as the distance between related information grew, the memory of that information deteriorated. This was a major bottleneck that stopped RNNs from scaling to the kind of language understanding we expect today.

Another major problem with RNNs and LSTM was the training was slow and expensive. Think of RNNs like workers passing bucket of water down the line. Each worker has to wait for the one before them.

In the same way, RNNs had to process each word step by step in a strict sequence. This created a massive bottleneck. You couldn't skip ahead or process multiple words at once.

Transformers changed the game completely. Instead of waiting in line, imagine all the workers fetching water at the same time. That's exactly what transformers do.

They process all words in parallel, dramatically increasing efficiency. This single difference in architecture meant transformers could be trained on much larger datasets in far less time. It unlocked the scaling power that drives today's massive AI systems like GPT and multimodal models.

Without this parallel processing ability, training modern models would take years, and cost far more than it does today. Transformers made the leap from small research models to industry-scale AI possible. If transformers had never been invented, the applications of AI would still be extremely limited.

Take translation services as an example. It would still be relying on weak statistical-based translators. They would make frequent errors, fail to handle idiomatic expressions, and often miss context-dependent meanings.

A sentence like kick the bucket might still be translated literally instead of as an idiom meaning to die. In conversational AI, chatbots would remain rigid and rule-based. Their response would be pre-programmed, offering no real understanding of nuance and no ability to generate original text.

Conversations would feel robotic, predictable, and very far from human-like interaction. For computer vision, progress would also be held back. System would still rely mainly on convolutional neural networks, which are powerful but limited.

They could classify objects, but they struggle to capture deeper relationship and context within an image, something that vision transformers now excel at. And perhaps the most important part, multimodal AI, systems that can understand text, images, and speech together. This would simply not exist.

There will be no unified model capable of connecting language with vision or audio in a meaningful way. So when we write a prompt to create an image and it easily creates an image, some of the applications, that's because of transformers. So in short, without transformers, we would still be living with clunky translators, scripted chatbots, basic vision system, and no multimodal intelligence at all.



So that's what transformer has done for you. And the path-breaking paper that has made all this possible came in 2017, Attention is All You Need, introduced the concept of transformers with revolutionary attention mechanism that we talked about, which provides superior context retention, even for the longer sequences when you're processing paragraphs or even pages. It allows you to do parallel processing, versatile architecture adoptable across domains, and foundation for all modern language models.

In fact, you need to understand this, that before transformers, when we were relying on RNNs for different tasks, we had different kind of architecture. Like if I am doing a language translation, we will go with many-to-many architecture, or if we are going for a sentiment analyser, we had many-to-one architecture, many inputs but one output. So for each task, we had a different architecture that we need to, we have to work upon.

But with transformers, that part was taken care of. A single architecture which could solve different kind of problems. Let's summarise what life would look like without transformers.

If transformers had never been invented, RNNs would still be the dominant models. Progress in AI would be advancing at only a fraction of today's pace, and breakthroughs would be far less frequent. The generative AI revolution, tools like ChatGPT, Dally, StableDiffusion, or Whisper simply wouldn't exist.

Instead of creative assistants, we would still be stuck with basic limited systems. AI systems in general would be less accurate, poorly scalable, and far more expensive to run. Training models would take much longer, and their performance would not justify the huge costs.

Most importantly, the adoption of AI in the real world would be about 10 years behind where we are today. Many of the tools we now take for granted, from advanced translation to realistic image generation, would still be in research labs, not in everyday use. In short, without transformers, the AI world would look smaller, slower, and far less impactful.

Attention is all you need, and without it, AI would not be where it is today. Transformers did not just improve AI; they redefined what was possible. They turned science fiction into science fact, from fluent conversations with ChatGPT, to instance translation across languages, to generating realistic images and even music.

Transformers unlocked capabilities that was once unimaginable. And that's what transformers have done for us.